AI Day Trader

Long Premium, Short Leash

An autonomous options agent on Alpaca's MCP server.
Alpaca AI Trading Agents Hackathon - Aug 28 to Sep 4, 2026

Bill McCormick - account `PA3VS39Y5LE2` - MIT

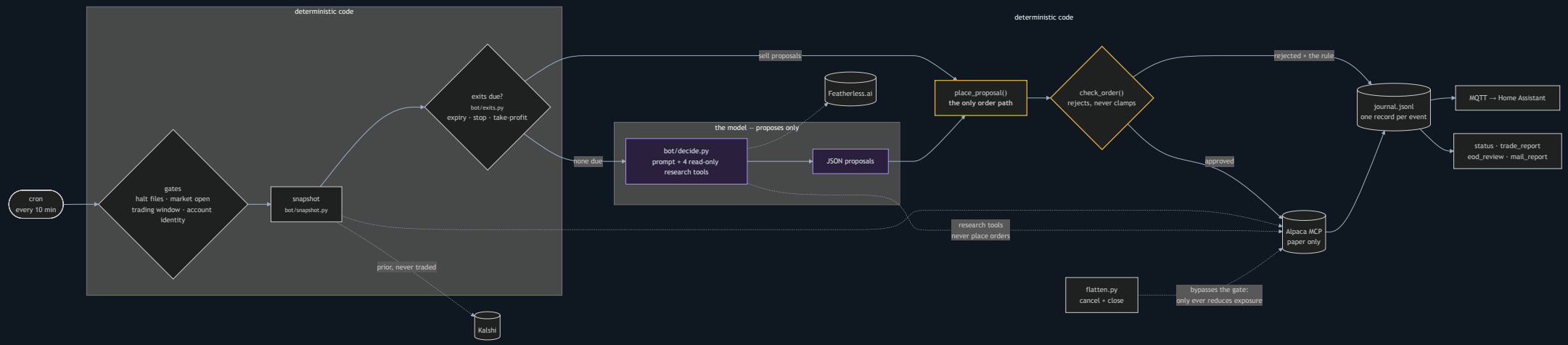
The thesis, in one sentence

Buy defined-risk, short-dated options premium on the most liquid names when an open-source model can *name a reason*; deterministic code sizes every trade, stops it, and closes it before expiry.

The model never touches an order. Open model proposes; code disposes.

Why long premium: the worst case is known before entry, which keeps every guardrail simple and absolute - and it matches what a language model is good at (a directional view from evidence) rather than what it is bad at (managing a position minute to minute).

One cycle, every 10 minutes



Outlined boxes are code only. There is no path from the model to an order.

Alpaca's free options feed has no Greeks

So the agent solves implied volatility from each contract's market price (Black-Scholes, bisection - robust far from the money) and derives delta, gamma, theta and vega on the fly.


```
{"symbol":"SPY260831P00769000","type":"put","strike":769.0,"dte":2,
 "bid":1.64,"ask":1.67,"last":1.67,
 "iv":0.0827,"delta":-0.4596,"gamma":0.0843,"theta":-0.4282,"vega":0.226}
```


Solved per contract from a noisy feed - so the prompt tells the model they are rough guides, not to audit them.

A second opinion from a prediction market

Kalshi's daily index-close markets are a crowd-implied distribution of today's close - a second, independent estimate the model can compare against what the option chain implies.

```
PREDICTION MARKETS (Kalshi, crowd-implied, read-only — a PRIOR to weigh, not
a signal to copy; compare to what the option chain implies and to today's
price action):
- SPY via KXINX: prior close 769; implied median 771.4 (+0.31%);
  P(above prior close) 0.58, P(up>1%) 0.21, P(down>1%) 0.14
```

Public API, no key, never traded - read-only, cached, and silently omitted on any failure.

The agent investigates before it acts

```
research: get_bars({'symbol': 'SPY', 'timeframe': '15Min'})        -> 3334
chars
research: get_stock_snapshot({'symbol': 'NVDA'})                   -> 622
chars
research: get_option_snapshot({'symbols': 'NVDA260902P00220000,...'}) ->
1339 chars
research: get_news({'symbols': 'NVDA', 'limit': 5})                -> 1770
chars
model output:
[{"instrument":"option","symbol":"NVDA260902P00220000","side":"buy",
  "qty":10,"order_type":"limit","limit_price":4.65,
  "reason":"NVDA is in a clear intraday downtrend (225 -> 218.6); the 220
put with
            4 DTE, delta -0.58, aligns with the bearish momentum"}]
```

Four read-only tools mapped to Alpaca MCP calls, six calls max, every one journaled. Nothing that orders is reachable.

Per proposal	whitelist - ≤ $5,000 per position - ≤ 4 positions - ≤ 10 contracts/order - 1-45 DTE - entries 09:45-15:15 ET
Per cycle, before the model	close on expiry day - stop-loss -40% - take-profit +60%
Per day	2% loss → flatten everything and halt - `HALT` file = global kill switch
End of day	close contracts expiring within a day; hold the rest under the stops (judged on Thursday's close)

```
REJECTED buy 12 SPY260904C00775000: qty 12 exceeds max_contracts_per_order
10
```

Stocks and options are both first-class. Every limit above applies to either one - the dollar cap counts `contracts x 100 x price` or share market value, and the position count is combined. The model states which instrument it wants and why: options when the edge is time-bounded, shares when volatility is expensive or the move is a slow

Everything is journaled

One JSONL record per event: `cycle_start`, `config` (hash + active overrides), `decision` (raw output, reasoning, usage, latency, tool calls), `tool_call`, `order_submitted` / `order_rejected` (with the rule), exits, halts.

The journal is what makes the loop evaluable - and what the end-of-day review reads.

It runs on our own hardware, not a laptop

A Proxmox LXC, provisioned by Ansible and driven by *cron* - not by anyone being awake. The judging account's keys never leave that container: root-only files the loader refuses to read from anywhere else.

A self-hosted GitHub Actions runner sits *on* that container, so a merge to `main` is deployed in about a minute - and a freeze window refuses to sync trading code between 08:20 and 15:15 CT, so live behaviour can never change mid-session.

And one guard keys on the credentials, not the CLI: every cycle checks the broker's own account number against the account it was asked for.

Built to be changed while it is running

A four-day agent is mostly an *iteration speed* problem: the constraint is not writing the strategy, it is changing it without breaking a live account.

495 tests, 1.3s	every one runs with **no API keys and no network** - clone and get a green suite before you have credentials
Only Docker needed	`docker compose run --rm bot` - tests, a full dry-run cycle, the docs site
Rehearse any time	`--dry-run` prints orders instead of sending; `--force` skips the market gates - a whole cycle is testable at 2am Sunday
Strategy changes without a deploy	`override.py` - allowlisted knobs, hard risk caps stay git-only, all of it expires at the close

Six of those tests exist because something broke - a report that published on one of four cycle paths, a test that passed only on a laptop. Each incident ends as a test, so the suite is a list of things that can no longer happen quietly.

The daily loop

16:05 `eod_review` → read: round trips, exit mix, rejections by rule, model's own read →

one change → `override.py` (expires at close) or config PR →

CI → self-hosted runner deploys before the open

Two more paper accounts run variant configs against the same live market - the only backtest available in four days. 75 PRs, 496 tests, every merge deployed automatically.

Four days isn't enough for a real backtest, and there's no historical options data on the free feed anyway. So: a docker-compose farm, one container per variant, each its own config and its own paper account, all trading the *same live market*.

```
$ docker compose --profile farm config --services
bot-kimi26

$ diff config.yaml config-variants/kimi26.yaml
< model: moonshotai/Kimi-K2-Instruct
> model: moonshotai/Kimi-K2.6
> research_tools_enabled: true
> learning_enabled: true
```

Different model, different thesis, different risk parameters - same conditions. One variant changes exactly one line: `mixed` gives stock and options equal footing instead of options-first, so "which instrument should this agent reach for" is answered by the P&L rather than by us.

HA auto-discovery: equity, day P&L, positions, halt, last decision, trades

```
alpaca-hackathon/official/event/order_submitted
{"side":"buy","symbol":"NVDA...","qty":10,...}
alpaca-hackathon/official/event/identity_refused {"reason":"resolved
credentials for the JUDGING account"}
alpaca-hackathon/official/state/equity           100240.50
[retained]
homeassistant/sensor/alpaca_official_halt/config {"device":{"name":"AI Day
Trader (official)"},...}
```

Three audiences, one feed. **Operator:** kill switch and tunable knobs. **Team:** a second, read-only dashboard over Tailscale - no switch, no button, because HA has no per-entity permissions, so the separation has to *be* the dashboard. **Phone:** push, but only for problems.

Fills deliberately do *not* push - a channel that fires on routine activity gets muted, and the halt alert goes with it. A stall detector does: no cycle

Results (fill Thu Sep 3 after the close)

Equity Mon open → Thu close: $___ → $___ (__%)

Round trips: __ - exits: __ stop / __ take-profit / __ expiry / __ model / __ flatten

Official vs challenger vs mixed instruments: __

What didn't work: __

What's next, and thanks

- Promote what the challenger proved: research tools, prediction-market prior, learning loop
- Multi-leg spreads once the gates cover assignment risk
- Home Assistant over MQTT: the light that goes red when the leash pulls

github.com/bill-mccormick-dg/alpaca-hackathon - MIT

Thanks: Alpaca, lablab.ai, Featherless AI